ADVANCED SQL QUERY SYSTEM

Objectives

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

1. Subqueries and Nested Queries

Definition:

A subquery is a query written inside another SQL query. The inner query is executed first, and its result is used by the outer query.

Types of Subqueries:

1. Single-row: Returns one value.
2. Multiple-row: Returns multiple values.
3. Nested: A query inside another query.
4. Correlated: Inner query depends on outer query.

Example: Find orders whose amount is greater than the average order amount.

```
SQL> SELECT ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT
  2  FROM ORDERS
  3  WHERE TOTAL_AMOUNT >
  4        (SELECT AVG(TOTAL_AMOUNT)
  5         FROM ORDERS);

  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
---------- ----------- ------------
      1001           1         2098
      1003           3         2598
      1004           4         1999
      1007           3         1999
```

2. Find Products Above Average Price

A subquery can be used to compare each product's price with the average product price.

```
SQL> SELECT
  2      PRODUCT_ID,
  3      PRODUCT_NAME,
  4      PRICE
  5  FROM PRODUCT
  6  WHERE PRICE >
  7        (SELECT AVG(PRICE)
  8         FROM PRODUCT);

PRODUCT_ID PRODUCT_NAME                                         PRICE
---------- -------------------------------------------------- ----------
       101 Laptop                                              55000
       102 Smartphone                                          25000
```

2.1 Products Below Average Price

```
SQL> SELECT
  2      PRODUCT_ID,
  3      PRODUCT_NAME,
  4      PRICE
  5  FROM PRODUCT
  6  WHERE PRICE <
  7        (SELECT AVG(PRICE)
  8         FROM PRODUCT);

PRODUCT_ID PRODUCT_NAME                                         PRICE
---------- -------------------------------------------------- ----------
       103 Headphones                                           2500
       104 Keyboard                                             1500
       105 Mouse                                                 800
       106 Monitor                                             12000
```

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
          1           3397
          2           1499
          3           4597
          4           1999
          5           1499
```

Step 2: Find the Maximum Purchase

```
SQL> SELECT MAX(TOTAL_PURCHASE)
  2  FROM
  3  (
  4      SELECT
  5          CUSTOMER_ID,
  6          SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  7      FROM ORDERS
  8      GROUP BY CUSTOMER_ID
  9  );

MAX(TOTAL_PURCHASE)
-------------------
               4597
```

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT
  2      C.ID,
  3      C.NAME,
  4      SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  5  FROM CUSTOMER C
  6  JOIN ORDERS O
  7      ON C.ID = O.CUSTOMER_ID
  8  GROUP BY C.ID, C.NAME
  9  HAVING SUM(O.TOTAL_AMOUNT) =
 10  (
 11      SELECT MAX(TOTAL_PURCHASE)
 12      FROM
 13      (
 14          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
 15          FROM ORDERS
 16          GROUP BY CUSTOMER_ID
 17      )
 18  );

        ID NAME                           TOTAL_PURCHASE
---------- ------------------------------ --------------
         3 kaushik                                  4597
```

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT
  2      C.ID,
  3      C.NAME,
  4      O.ORDER_ID,
  5      O.TOTAL_AMOUNT
  6  FROM CUSTOMER C
  7  JOIN ORDERS O
  8      ON C.ID = O.CUSTOMER_ID
  9  WHERE O.TOTAL_AMOUNT >
 10        (SELECT AVG(TOTAL_AMOUNT)
 11         FROM ORDERS);

        ID NAME                           ORDER_ID TOTAL_AMOUNT
---------- ------------------------------ ---------- ------------
         1 Ramesh                               1001         2098
         3 kaushik                              1003         2598
         3 kaushik                              1007         1999
         4 Chaitali                             1004         1999
```

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from ORDER_DETAILS.

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      SUM(OD.SUBTOTAL) AS TOTAL_SALES
  5  FROM PRODUCT P
  6  JOIN ORDER_DETAILS OD
  7      ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY
  9      P.PRODUCT_ID,
 10      P.PRODUCT_NAME
 11  HAVING SUM(OD.SUBTOTAL) >
 12  (
 13      SELECT AVG(PRODUCT_SALES)
 14      FROM
 15      (
 16          SELECT SUM(SUBTOTAL) AS PRODUCT_SALES
 17          FROM ORDER_DETAILS
 18          GROUP BY PRODUCT_ID
 19      )
 20  )
 21  ORDER BY TOTAL_SALES DESC;

PRODUCT_ID PRODUCT_NAME                                       TOTAL_SALES
---------- -------------------------------------------------- -----------
       103 Headphones                                                 700
       105 Mouse                                                      500
       104 Keyboard                                                   450
       101 Laptop                                                     390
```

6. Customer Purchase Report

A detailed customer purchase report can combine Customer and Orders tables.

```
SQL> SELECT
  2      C.ID,
  3      C.NAME,
  4      COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS,
  5      SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE,
  6      AVG(O.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  7  FROM CUSTOMER C
  8  JOIN ORDERS O
  9      ON C.ID = O.CUSTOMER_ID
 10  GROUP BY
 11      C.ID,
 12      C.NAME
 13  ORDER BY TOTAL_PURCHASE DESC;

        ID NAME                           NUMBER_OF_ORDERS TOTAL_PURCHASE
---------- ------------------------------ ---------------- --------------
AVERAGE_ORDER_VALUE
-------------------
         3 kaushik                                       2           4597
            2298.5

         1 Ramesh                                        2           3397
            1698.5

         4 Chaitali                                      1           1999
              1999


        ID NAME                           NUMBER_OF_ORDERS TOTAL_PURCHASE
---------- ------------------------------ ---------------- --------------
AVERAGE_ORDER_VALUE
-------------------
         2 Khilan                                        1           1499
              1499

         5 Hardik                                        2           1499
             749.5
```

7. Advanced Product Sales Report

This report combines Product and Order_Details tables.

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      COUNT(DISTINCT OD.ORDER_ID) AS NUMBER_OF_ORDERS,
  5      SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  6      SUM(OD.SUBTOTAL) AS TOTAL_SALES,
  7      AVG(OD.UNIT_PRICE) AS AVERAGE_SELLING_PRICE
  8  FROM PRODUCT P
  9  JOIN ORDER_DETAILS OD
 10      ON P.PRODUCT_ID = OD.PRODUCT_ID
 11  GROUP BY
 12      P.PRODUCT_ID,
 13      P.PRODUCT_NAME
 14  ORDER BY TOTAL_SALES DESC;
```

```
PRODUCT_ID PRODUCT_NAME                                        NUMBER_OF_ORDERS
---------- -------------------------------------------------- ----------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------------- ----------- ---------------------
       103 Headphones                                                        1
                   2         700                     350

       105 Mouse                                                             1
                   2         500                     250

       104 Keyboard                                                          1
                   1         450                     450

PRODUCT_ID PRODUCT_NAME                                        NUMBER_OF_ORDERS
---------- -------------------------------------------------- ----------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------------- ----------- ---------------------
       101 Laptop                                                            1
                   3         390                     130

       102 Smartphone                                                        1
                   1         180                     180

       106 Monitor                                                           1
                   1         120                     120

6 rows selected.
```

8. Category-wise Advanced Sales Report

Category-wise sales can be generated by combining Category, Product, and Order_Details.

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      COUNT(DISTINCT OD.ORDER_ID) AS NUMBER_OF_ORDERS,
  5      SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  6      SUM(OD.SUBTOTAL) AS TOTAL_SALES,
  7      AVG(OD.UNIT_PRICE) AS AVERAGE_SELLING_PRICE
  8  FROM CATEGORY C
  9  JOIN PRODUCT P
 10      ON C.CATEGORY_ID = P.CATEGORY_ID
 11  JOIN ORDER_DETAILS OD
 12      ON P.PRODUCT_ID = OD.PRODUCT_ID
 13  GROUP BY
 14      C.CATEGORY_ID,
 15      C.CATEGORY_NAME
 16  ORDER BY TOTAL_SALES DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
---------------- ------------------- ----------- ---------------------
          1
Mobile Accessories
               3                   6        1830                 307.5

          3
Home Appliances
               2                   4         510                  125
```